Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И УПРАВЛЕНИЕ» (ИУК)

КАФЕДРА «СИСТЕМЫ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ» (ИУК3)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме

Студент группы ИУК3-71Б — А.А. Исаев

(подпись, дата) (И.О. Фамилия)

Руководитель курсового проекта — А.М. Макаренков

(подпись, дата) (И.О. Фамилия)

Калуга, 2026

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУК-3
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине _____
Студент группы ИУК3-71Б Исаев Артём Алексеевич
Тема курсового проекта: **Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме**
Направленность КП: **учебный**
Источник тематики: **кафедра ИУК-3**

Задание

– Описать модели типовых звеньев в пространстве состояний и способы перехода от структурной схемы к модели всей системы.
– Разработать алгоритм построения матриц A, B, C, D по структурной схеме, включая схемы с подсистемами и петлями без инерционных звеньев.
– Реализовать подсистему в серверной части системы моделирования и вывести полученную модель в веб-интерфейсе.
– Проверить подсистему на контуре с ПИ-регулятором, схеме с перекрёстными обратными связями и электрогидравлическом следящем приводе 8-го порядка: сравнить с моделью той же схемы в Simulink и расчётом в MATLAB.

Оформление курсового проекта

Расчетно-пояснительная записка на _____ листах формата А4.
Перечень графического материала КП (плакаты, схемы, чертежи и т.п.):
– схема алгоритма построения модели – 1 лист формата А3;
– примеры построения модели и результаты проверки – 1 лист формата А3.

Дата выдачи задания «___» _____ 2026 г.

Руководитель _____ А.М. Макаренков
(подпись, дата) (И.О. Фамилия)

Студент _____ А.А. Исаев
(подпись, дата) (И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине _____

Студент группы ИУК3-71Б Исаев Артём Алексеевич

Тема курсового проекта: **Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме**

№	Наименование этапов	Срок, план	Срок, факт	Отметка руководителя	Куратор
1	Задание на выполнение КП	3-я нед.			
2	Изучение моделей звеньев и способов построения модели схемы	6-я нед.			
3	Разработка алгоритма и реализация подсистемы	10-я нед.			
4	Верификация на примерах в Simulink и MATLAB	13-я нед.			
5	Оформление расчетно-пояснительной записки и графической части	15-я нед.			
6	Защита	17-я нед.			

Студент _____
(подпись, дата)

Руководитель _____
(подпись, дата)

Содержание

Перечень сокращений и обозначений.....7
Введение.....8
Постановка задачи.....9
1 Модель системы в пространстве состояний.....10
1.1 Что такое модель в пространстве состояний.....10
1.2 Модели типовых звеньев.....11
1.3 Как получить модель всей схемы.....12
2 Алгоритм построения модели.....14
2.1 Общий порядок работы.....14
2.2 Проверка схемы.....14
2.3 Раскрытие подсистем.....14
2.4 Объединение уравнений звеньев.....15
2.5 Таблицы соединений.....15
2.6 Итоговые матрицы и петли без инерционных звеньев.....15
3 Примеры построения модели.....18
3.1 Контур с ПИ-регулятором.....18
3.2 Перекрёстные обратные связи.....19
3.3 Электрогидравлический следящий привод 8-го порядка.....20
4 Программная реализация.....25
4.1 Место подсистемы в системе моделирования.....25
4.2 Входные данные.....25
4.3 Основные структуры данных.....26
4.4 Функции построения модели.....26
4.5 Что получает пользователь.....27
5 Верификация в Simulink и MATLAB.....28
5.1 Как проверяется модель.....28
5.2 Результаты.....30
5.3 Осциллограммы.....31
6 Руководство пользователя.....33
6.1 Построение модели схемы.....33
6.2 Работа с подсистемами.....33
6.3 Если модель не построена.....34
Заключение.....35
Список литературы.....36

Приложение А. Главный алгоритм.....37
Приложение Б. Расчёт ЭГСП в MATLAB.....39

Перечень сокращений и обозначений

САУ – система автоматического управления

ПФ – передаточная функция

x, u, y – векторы состояния, входов и выходов

A, B, C, D – матрицы модели $\dot{x} = Ax + Bu,\ y = Cx + Du$

M, N – таблицы (матрицы) соединений схемы

κ – число обусловленности петли без инерционных звеньев

ЭГСП – электрогидравлический следящий привод

ЭМП – электромеханический преобразователь

ЭГУ – электрогидравлический усилитель

ГЦ – гидроцилиндр

JSON – текстовый формат, в котором схема передаётся на сервер

Введение

Систему автоматического управления (САУ) обычно рисуют структурной схемой. Каждое звено на ней изображается прямоугольником со своей передаточной функцией, а стрелки показывают, куда идёт сигнал. Человеку такая схема понятна сразу, но для расчёта её нужно превратить в уравнения.

В программах моделирования это делается двумя способами. В первом блоки считаются по очереди на каждом шаге времени: каждый блок по своему входу находит свой выход и скорость изменения своих переменных. Так устроено моделирование в Simulink [10]. Уравнения всей системы при этом не составляются. Чтобы получить полюса или частотные характеристики, схему в Simulink приходится отдельно линеаризовать, например функцией linmod [10].

Во втором способе вся схема один раз превращается в одну систему уравнений с матрицами A, B, C, D (модель в пространстве состояний). По ней сразу считаются и переходные процессы, и полюса, и передаточная функция, и частотные характеристики. Трудность в том, чтобы построить эти матрицы для произвольной схемы: с несколькими контурами, с подсистемами внутри подсистем и с петлями, по которым сигнал проходит без задержки.

Курсовой проект является частью выпускной квалификационной работы «Веб-ориентированная система визуального моделирования систем автоматического управления» (Control Lab). В нём разработана подсистема, которая по структурной схеме, собранной пользователем в браузере, строит матрицы A, B, C, D всей системы и показывает, какой блок схемы отвечает за каждую переменную модели. Остальные расчёты Control Lab выполняются уже по этой модели.

Постановка задачи

В курсовом проекте требуется:

1. Описать, как типовые звенья записываются в пространстве состояний, и сравнить способы перехода от структурной схемы к модели всей системы.

2. Разработать алгоритм, который строит матрицы A, B, C, D по схеме с обратными связями, подсистемами и петлями без инерционных звеньев.

3. Выдавать сообщение с причиной и списком блоков для схем, модель которых построить нельзя или нельзя построить с достаточной точностью.

4. Реализовать подсистему в серверной части Control Lab (Python) и показать полученную модель во вкладке «Матрицы» веб-интерфейса.

5. Разобрать работу алгоритма на трёх примерах (контур с ПИ-регулятором, перекрёстные обратные связи, электрогидравлический следящий привод 8-го порядка) и проверить результат: построить те же схемы в Simulink и сравнить осциллограммы, а передаточные функции сравнить с расчётом в MATLAB.

1 Модель системы в пространстве состояний

1.1 Что такое модель в пространстве состояний

Любое линейное звено можно представить как устройство со входом u и выходом y (рисунок 1). Если звено инерционное, его выход зависит не только от входа в данный момент, но и от того, что происходило раньше. Например, интегратор накапливает входной сигнал, а апериодическое звено догоняет вход с задержкой. Величины, которые звено «помнит», называют переменными состояния и обозначают x.



Рисунок 1 – Звено: вход u, состояние x, выход y

Модель в пространстве состояний состоит из двух уравнений [2, 3]:

$$\dot{x} = A\,x + B\,u\,,\; y = C\,x + D\,u\,. \qquad (1)$$

Первое уравнение задаёт скорость изменения состояния ($\dot{x}$ означает производную по времени), второе показывает, как из состояния и входа получается выход. Матрицы A, B, C, D — это таблицы коэффициентов этих уравнений:

– A описывает, как состояние влияет само на себя (например, затухание);

– B описывает, как вход меняет состояние;

– C показывает, какая комбинация переменных состояния видна на выходе;

– D показывает, какая часть входа сразу, без задержки, проходит на выход.

Если такая модель построена для всей схемы, из неё получаются основные характеристики системы [2]:

– полюса — собственные числа матрицы A, то есть корни уравнения $\det(sI - A) = 0$; система устойчива, если у всех полюсов вещественная часть отрицательна;

– передаточная функция $W(s) = C(sI - A)^{-1}B + D$;

– переходный процесс — решение первого уравнения при ступенчатом входе;

– частотная характеристика $W(j\omega)$.

Поэтому все расчёты Control Lab опираются на модель схемы в пространстве состояний.

1.2 Модели типовых звеньев

Для каждого звена матрицы A_i, B_i, C_i, D_i выписываются по его параметрам. Покажем это на апериодическом звене $W(s) = \frac{K}{Ts + 1}$. Во времени ему соответствует уравнение $T\dot{y} + y = Ku$. Если взять в качестве состояния сам выход, $x = y$, получим

$$\dot{x} = -\frac{1}{T}x + \frac{K}{T}u,\ y = x, \tag{2}$$

то есть $A = -1/T$, $B = K/T$, $C = 1$, $D = 0$. У интегратора $\frac{k}{s}$ состоянием служит накопленное значение: $A = 0$, $B = k$, $C = 1$, $D = 0$. У усилителя и сумматора состояний нет, выход появляется сразу: у усилителя $D = k$, у сумматора D — строка из $+1$ и -1 по знакам входов. Звено с произвольной передаточной функцией $\frac{N(s)}{D(s)}$ записывается в канонической форме [2]; число его состояний равно степени знаменателя. Все звенья, которые поддерживает Control Lab, приведены в таблице 1.

Таблица 1 – Типовые звенья и их модели

Звено	**Передаточная функция**	**Состояний**	**Мгновенная передача**
Усилитель	k	0	да
Сумматор	$\pm u_1 \pm u_2 \pm \dots$	0	да
Интегратор	$\frac{k}{s}$	1	нет
Звено 1-го порядка	$\frac{K}{T s + 1}$	1	нет
Колебательное звено	$\frac{K \omega_0^2}{s^2 + 2\zeta \omega_0 s + \omega_0^2}$	2	нет
Передаточная функция	$\frac{N(s)}{D(s)}$	степень D(s)	если степени N и D равны
ФНЧ Баттерворта	порядок n, частота среза ω_c	n	нет
ПИД-регулятор	$K_p + \frac{K_i}{s} + \frac{K_d N s}{s + N}$	2 (1 при $K_d = 0$)	да

В последнем столбце указано, передаёт ли звено изменение входа на выход сразу ($D_i \neq 0$). Это понадобится в разделе 2.6. Если сигнал по петле проходит только через такие звенья, он обходит её мгновенно, и выходы блоков петли приходится находить, решая систему уравнений.

1.3 Как получить модель всей схемы

Способы получить модель всей схемы сравниваются в таблице 2.

Таблица 2 – Способы получить модель всей схемы

Способ	**Как работает**	**Ограничения**
Правила структурных преобразований (вручную)	Последовательные звенья перемножают, параллельные складывают, контур сворачивают по формуле $W/(1+W)$ [1]	Долго и легко ошибиться, если контуров несколько; даёт только передаточную функцию
Поблочный расчёт (первая версия Control Lab)	На каждом шаге времени блоки считаются по очереди, по ходу сигнала	Нет матриц всей системы; петлю без инерционных звеньев посчитать нельзя
Построение единой модели (разработано в проекте)	Уравнения всех звеньев объединяются, соединения подставляются, получаются A, B, C, D	Только линейные звенья

Первая версия Control Lab считала схему поблочно. Такой расчёт не даёт матриц всей системы, а петлю, по которой сигнал проходит без

задержки, им посчитать нельзя: чтобы найти выход первого блока петли, нужен выход последнего, а он сам зависит от первого. Поэтому в проекте разработан третий способ. Ему не нужно искать на схеме контуры и выбирать порядок их сворачивания: каждое соединение записывается одной единицей в таблице соединений (раздел 2.5).

2 Алгоритм построения модели

2.1 Общий порядок работы

Алгоритм состоит из шести шагов. Они описаны в разделах 2.2–2.6, причём раздел 2.6 объединяет два последних шага: решение уравнений выходов и вычисление итоговых матриц. Схема алгоритма и текст главной функции приведены в приложении А. Если на каком-то шаге модель построить нельзя, пользователь получает сообщение с причиной и названиями блоков, которые нужно исправить.

2.2 Проверка схемы

Схема приходит на сервер в виде списка блоков и списка соединений «выход блока → вход блока» (пример приведён в разделе 4.2). Сначала проверяется, что схему можно рассчитать: все блоки известного типа, параметры допустимы (например, постоянная времени $T>0$), к каждому входу подключён ровно один сигнал, нет соединений с несуществующими портами. Найденные ошибки собираются в один список, чтобы пользователь увидел их все сразу, а не по одной.

2.3 Раскрытие подсистем

Подсистема — это блок, внутри которого своя схема со своими входами и выходами. Для расчёта она не нужна как отдельный объект, поэтому алгоритм раскрывает подсистемы: переносит их блоки на верхний уровень, а связи, проходившие через границу подсистемы, соединяет напрямую. Вложенные подсистемы раскрываются так же, по очереди сверху вниз.

Чтобы имена блоков из разных подсистем не совпали, к имени каждого блока добавляется путь, например measurement::sensor_path::filter. Поэтому две копии одной подсистемы не мешают друг другу, а пользователь видит, из какой подсистемы пришла каждая переменная модели. Если выход подсистемы соединён прямо с её входом, сигнал проходит её насквозь.

2.4 Объединение уравнений звеньев

После раскрытия у каждого звена i есть свои матрицы A_i, B_i, C_i, D_i (раздел 1.2). Запишем уравнения всех звеньев вместе, пока без соединений. Состояния всех звеньев собираются в один вектор x, все входы — в вектор u_b, все выходы — в вектор y_b. Матрицы звеньев ставятся по диагонали общей матрицы, остальные места заполняются нулями:

$$A_d = \begin{pmatrix} A_1 & 0 & \cdots \\ 0 & A_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}, \dot{x} = A_d x + B_d u_b,\ y_b = C_d x + D_d u_b. \tag{3}$$

Матрицы B_d, C_d, D_d составляются так же. Пока это описание звеньев, которые ещё не связаны.

2.5 Таблицы соединений

Теперь нужно учесть соединения. На каждый вход приходит ровно один сигнал: либо выход другого блока, либо внешнее воздействие r (например, ступенчатый вход). Это записывается двумя таблицами из нулей и единиц:

$$u_b = M\, y_b + N\, r. \tag{4}$$

В таблице M строки соответствуют входам блоков, столбцы — выходам. Единица в строке «вход усилителя» и столбце «выход сумматора» означает, что выход сумматора подключён ко входу усилителя. Таблица N так же показывает, к каким входам подключены внешние воздействия. Сигналы, которые видят осциллографы, записываются так же: $y = S_y y_b + S_r r$. Все таблицы заполняются автоматически, за один проход по списку соединений. Пример заполненной таблицы приведён в разделе 3.1.

2.6 Итоговые матрицы и петли без инерционных звеньев

Подставим соединения $u_b = M\, y_b + N\, r$ в уравнение выходов звеньев:

$$\left(I - D_d M\right) y_b = C_d x + D_d N\, r, \tag{5}$$

где I — единичная матрица. Это система линейных уравнений относительно выходов y_b. Её решение выражает выходы всех блоков через состояние и воздействия: $y_b = Y_x x + Y_r r$. Подставив его в уравнения состояния и осциллографов, получаем модель всей схемы:

$$A = A_d + B_d M Y_x,\ B = B_d\left(M Y_r + N\right),\ C = S_y Y_x,\ D = S_y Y_r + S_r. \tag{6}$$

Решать систему целиком обычно не требуется. В большинстве схем сигнал идёт от блока к блоку, и выходы находятся по очереди подстановкой уже найденных. Поэтому алгоритм сначала выделяет группы блоков, по которым сигнал может обойти круг мгновенно, через звенья с $D_i \neq 0$. На языке теории графов это сильно связные компоненты графа мгновенных передач. Они ищутся функцией connected_components библиотеки SciPy [7], которая реализует вариант алгоритма Тарьяна [5, 11]. Выходы остальных блоков находятся подстановкой, а систему уравнений приходится решать только внутри найденных групп.

Такая группа называется алгебраической петлёй. Решение петли бывает очень чувствительным к коэффициентам. Простейший пример — усилитель k, охваченный положительной обратной связью:

$$y = k\left(u + y\right) \Rightarrow y = \frac{k}{1-k} u. \tag{7}$$

При $k = 1$ решения нет. При k, близком к единице, решение есть, но небольшая неточность в k сильно меняет результат: при $k = 0{,}999$ получаем $y = 999\,u$, а при $k = 0{,}998$ уже $y \approx 499\,u$. Изменение коэффициента на 0,1 % изменило ответ вдвое.

Насколько петля усиливает погрешности, показывает число обусловленности κ: относительная погрешность решения может быть в κ раз больше относительной погрешности коэффициентов [4]. Для петли из нескольких блоков обозначим через G матрицу мгновенных передач внутри петли, а через $\sigma_{min}\left(I - G\right)$ наименьшее сингулярное число матрицы $I - G$. Подсистема использует верхнюю оценку числа обусловленности матрицы $I - G$ [4]:

$$\kappa = \frac{1 + \| G \|}{\sigma_{min}(I - G)}, \text{ для одного усилителя } \kappa = \frac{1 + |k|}{|1 - k|}. \quad (8)$$

Компьютер хранит числа с относительной погрешностью $\varepsilon \approx 2,2 \cdot 10^{-16}$, то есть примерно с 16 верными знаками. Погрешность решения петли оценивается как $\kappa\varepsilon$ [4]. Если $\kappa > 1/\sqrt{\varepsilon} \approx 6,7 \cdot 10^{7}$, эта погрешность больше $\sqrt{\varepsilon} \approx 1,5 \cdot 10^{-8}$, и в ответе остаётся меньше половины верных знаков. Такая погрешность уже сравнима с погрешностью расчёта переходного процесса (10^{-8}, раздел 5.1), а при большем κ превышает её, и результаты моделирования перестают быть достоверными. Поэтому в этом случае подсистема не строит модель. Она сообщает, какие блоки образуют петлю, и предлагает изменить коэффициенты или добавить в петлю инерционное звено.

3 Примеры построения модели

Работа алгоритма показана на трёх схемах возрастающей сложности: контур с ПИ-регулятором, схема с перекрёстными обратными связями и электрогидравлический следящий привод (ЭГСП) 8-го порядка. Для первых двух модель построена вручную по шагам главы 2 и сравнена с ответом программы. Модели всех трёх схем проверены в главе 5: те же схемы собраны в Simulink, а передаточные функции посчитаны в MATLAB. Все три схемы есть в Control Lab как готовые примеры.

3.1 Контур с ПИ-регулятором

Схема показана на рисунке 2. ПИ-регулятор $W_p(s)=K_п+K_и/s$ с $K_п=2$, $K_и=1$ управляет объектом из двух инерционных звеньев $\frac{1}{s+1}$ и $\frac{1}{0,5s+1}$. Обратная связь единичная отрицательная. В Control Lab регулятор задан блоком «ПИД-регулятор» с $K_д=0$.

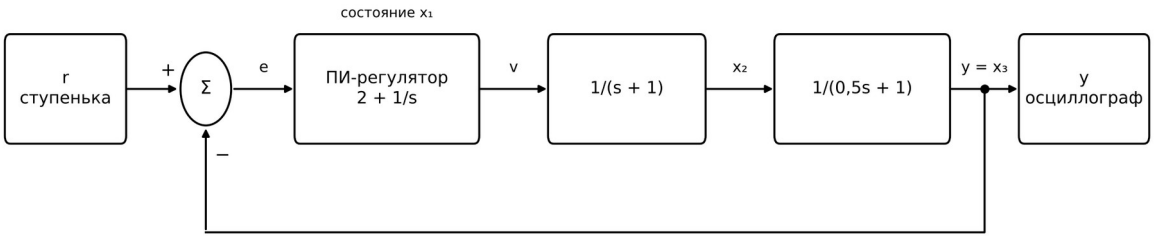


Рисунок 2 – Контур с ПИ-регулятором

Шаг 1. Уравнения блоков. У регулятора одно состояние x_1, накопленная ошибка: $\dot{x}_1=e$, выход $v=K_и x_1+K_п e=x_1+2e$. Уравнения звеньев (раздел 1.2):

$$\dot{x}_2=-x_2+u_2,\dot{x}_3=-2x_3+2u_3,y=x_3. \tag{9}$$

Шаг 2. Соединения приведены в таблице 3. Последний столбец — таблица N: ступенька подключена ко входу «+» сумматора.

Таблица 3 – Таблица соединений контура с ПИ-регулятором

Вход блока	**выход сумматора e**	**выход регулятора v**	**выход звена 1**	**выход звена 2**	**r (N)**
сумматор «+»	0	0	0	0	1
сумматор «−»	0	0	0	1	0
регулятор	1	0	0	0	0
звено 1	0	1	0	0	0
звено 2	0	0	1	0	0

Шаг 3. Подстановка по ходу сигнала:

$$e=r-x_3,\ v=x_1+2(r-x_3)=x_1-2x_3+2r, \tag{10}$$

$$\dot{x}_1=-x_3+r,\ \dot{x}_2=x_1-x_2-2x_3+2r,\ \dot{x}_3=2x_2-2x_3. \tag{11}$$

Шаг 4. Матрицы составляются из коэффициентов при x_1, x_2, x_3 и r:

$$A=\begin{pmatrix}0 & 0 & -1\\ 1 & -1 & -2\\ 0 & 2 & -2\end{pmatrix},\ B=\begin{pmatrix}1\\2\\0\end{pmatrix},\ C=\begin{pmatrix}0 & 0 & 1\end{pmatrix},\ D=0. \tag{12}$$

Программа выдала те же матрицы. **Шаг 5. Проверка.** По правилам структурных преобразований [1] $W=\dfrac{W_p W_1 W_2}{1+W_p W_1 W_2}=\dfrac{4s+2}{s^3+3s^2+6s+2}$. Характеристический многочлен матрицы A равен $det(sI-A)=s^3+3s^2+6s+2$, то есть совпадает со знаменателем W. Установившееся значение $W(0)=2/2=1$: интегральная часть регулятора убирает статическую ошибку.

3.2 Перекрёстные обратные связи

На рисунке 3 два местных контура перекрываются. Обратная связь H_1 идёт с выхода G_2 на вход G_1, а H_2 — с выхода G_3 на вход G_2. Звено G_2 входит в оба контура, но ни один из них не лежит внутри другого. Поэтому правила последовательного, параллельного и встречного соединения сразу не применяются: сначала нужно перенести точку съёма сигнала через звено и только потом сворачивать схему [1]. Для алгоритма программы эта схема не сложнее предыдущей: каждая обратная связь даёт одну единицу в таблице M.

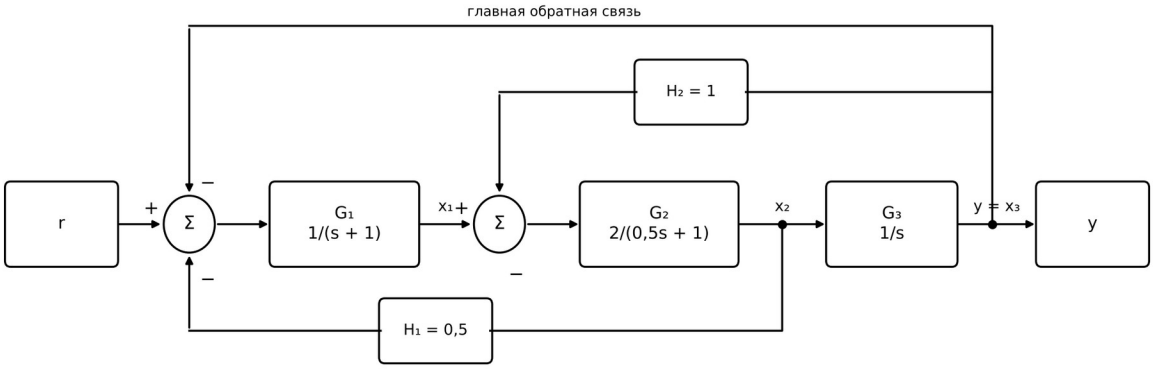


Рисунок 3 – Схема с перекрёстными обратными связями

Шаги 1–3. Уравнения звеньев и входы после подстановки соединений:

$$\dot{x}_1 = -x_1 + u_1,\ \dot{x}_2 = -2x_2 + 4u_2,\ \dot{x}_3 = u_3, \tag{13}$$

$$u_1 = r - x_3 - 0{,}5x_2,\ u_2 = x_1 - x_3,\ u_3 = x_2. \tag{14}$$

Шаг 4. Матрицы:

$$A = \begin{pmatrix} -1 & -0{,}5 & -1 \\ 4 & -2 & -4 \\ 0 & 1 & 0 \end{pmatrix},\ B = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix},\ C = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix},\ D = 0. \tag{15}$$

Шаг 5. Проверка по формуле Мейсона [12]. Прямой путь один: $P_1 = G_1G_2G_3$. Контуров три: $L_1 = -G_1G_2H_1$, $L_2 = -G_2G_3H_2$ и главный $L_3 = -G_1G_2G_3$. Все они попарно касаются, потому что проходят через G_2. Значит, $\Delta = 1 - L_1 - L_2 - L_3$, а $\Delta_1 = 1$:

$$W = \frac{G_1G_2G_3}{1 + G_1G_2H_1 + G_2G_3H_2 + G_1G_2G_3} = \frac{4}{s^3 + 3s^2 + 8s + 8}. \tag{16}$$

Характеристический многочлен матрицы A программы $\det(sI - A) = s^3 + 3s^2 + 8s + 8$ совпадает со знаменателем. Установившееся значение $W(0) = 0{,}5$.

3.3 Электрогидравлический следящий привод 8-го порядка

ЭГСП — привод, в котором слабый электрический сигнал управляет мощным гидроцилиндром. Модель и значения параметров взяты из заданий к практическим занятиям по дисциплине «Исполнительные устройства систем управления» [8].

Работает привод так. Напряжение U через усилитель подаётся на обмотку электромеханического преобразователя (ЭМП). Якорь ЭМП поворачивает заслонку, из-за этого меняется давление на торцах золотника, и золотник смещается. Золотник открывает путь маслу в гидроцилиндр (ГЦ), и поршень начинает двигаться. Датчик перемещения поршня возвращает сигнал на вход, и поршень останавливается, когда этот сигнал уравновешивает входное напряжение.

В [8] модель записана уравнениями (1.1)–(1.6) с двумя упрощениями. Первое: не учитывается сила, с которой струя масла давит на заслонку ($k_{Mp}=0$). Второе: масло в управляющих полостях считается несжимаемым, потому что перепад давлений там небольшой. Согласно [8], оба упрощения почти не влияют на поведение привода. После упрощений модель выглядит так:

$$L\dot{i}+R_a i=u,\; u=k_{yc}\left(U-k_{oc}Y\right), \tag{17}$$

$$J_я\ddot{\varphi}+h_я\dot{\varphi}+k_{M\varphi}\varphi=k_{Mi}i-k_{мос}x, \tag{18}$$

$$k_{Q_y p_y}p_y=k_{Q_y\varphi}\varphi-F_з\dot{x},\; m_з\ddot{x}+h_з\dot{x}+2C_{гд}x=F_з p_y, \tag{19}$$

$$\frac{V}{2E}\dot{p}+k_{Qp}p=k_{Qx}x-F\dot{Y},\; M\ddot{Y}+h\dot{Y}+CY=Fp. \tag{20}$$

Здесь i — ток обмотки, φ — угол поворота якоря, x — перемещение золотника, p_y и p — перепады давлений на золотнике и на поршне, Y — перемещение поршня. Значения параметров приведены в таблице 4. В задании к занятию 01 модуль упругости масла E указан в МПа; по указанию преподавателя единица исправлена на Па, то есть $E=10^7$ Па.

Таблица 4 – Параметры ЭГСП

Параметр	Значение	Параметр	Значение
$k_{ус}$	50	$k_{Q_y\varphi}$	10^{-3} м³/с
L, R_a	0,4 Гн; 80 Ом	$k_{Q_y p_y}$	10^{-6} м⁴·с/кг
$J_я$	$0{,}3\cdot10^{-6}$ кг·м²	$F_з$	$0{,}785\cdot10^{-4}$ м²
$h_я$	$7\cdot10^{-5}$ Н·м·с	$m_з$, $h_з$	0,005 кг; 0,25 Н·с/м
$k_{M\varphi}$, k_{Mi}	0,1 Н·м; 1 Н·м/А	$C_{гд}$	1 Н/м
$k_{мос}$	10 Н	k_{Qx}	4,3 м²/с
V, E	$647\cdot10^{-6}$ м³; 10^7 Па	k_{Qp}	10^{-10} м⁴·с/кг
M, h, C	10 кг; 10 Н·с/м; 0	F	$1{,}99\cdot10^{-3}$ м²
$k_{ос}$	0,1 В/м	U	0,01 В (ступенька)

Структурная схема модели показана на рисунке 4. В ней четыре обратные связи, вложенные одна в другую: по скорости золотника ($F_з\dot{x}$), механическая ($k_{мос}x$), по скорости поршня ($F\dot{Y}$) и главная электрическая ($k_{ос}Y$).

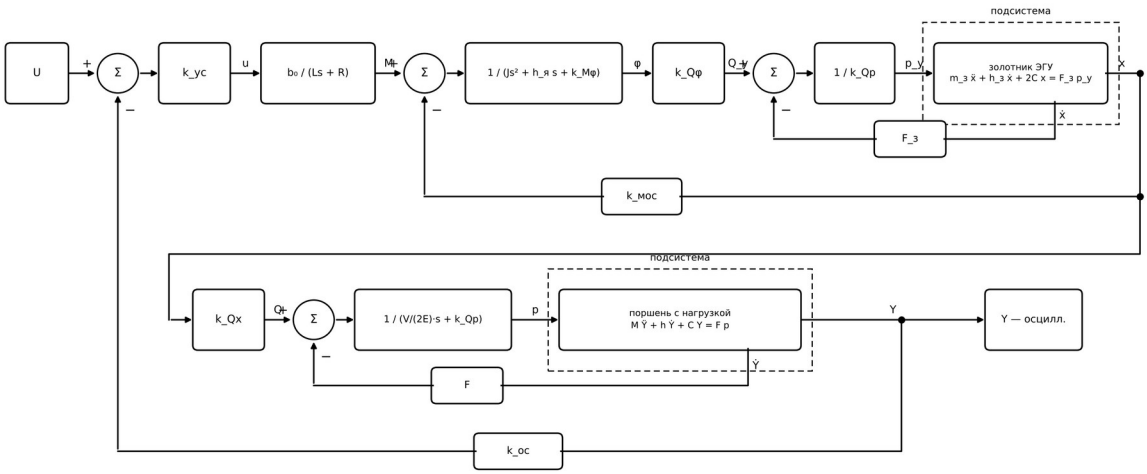


Рисунок 4 – Структурная схема ЭГСП

Как схема собрана в программе. В [8] обратные связи по скорости нарисованы звеньями $F_з s$ и $F s$, то есть идеальными дифференциаторами. Такое звено нельзя записать в пространстве состояний: у него степень числителя больше степени знаменателя, и для его выхода потребовалась бы производная входного сигнала. Но скорость $\dot{x}$ уже есть внутри модели золотника, её не нужно получать дифференцированием. Поэтому золотник и поршень собраны как подсистемы из сумматора, усилителей и двух интеграторов: ускорение интегрируется в скорость, скорость — в положение.

У каждой такой подсистемы два выхода, положение и скорость (рисунок 5). Уравнения при этом остаются теми же, что в [8], меняется только способ их записи. Остальные звенья заданы передаточными функциями. На верхнем уровне схемы 19 блоков, после раскрытия подсистем — 31.

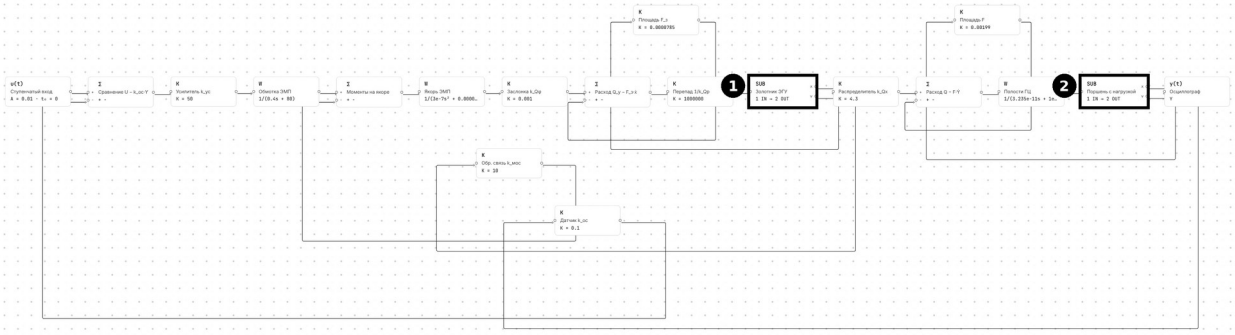


Рисунок 5 – ЭГСП в Control Lab: (1) подсистема «Золотник ЭГУ», (2) подсистема «Поршень с нагрузкой»

Программа построила модель 8-го порядка; её переменные состояния перечислены в таблице 5.

Таблица 5 – Переменные состояния модели ЭГСП

x	**Блок**	**Физический смысл**
x_1	обмотка ЭМП	ток i, умноженный на постоянный коэффициент
x_2, x_3	якорь ЭМП	скорость и угол поворота якоря (каноническая форма звена 2-го порядка)
x_4	золотник :: скорость	скорость золотника ẋ
x_5	золотник :: положение	перемещение золотника x
x_6	полости ГЦ	перепад давлений p, умноженный на постоянный коэффициент
x_7	поршень :: скорость	скорость поршня Ẏ
x_8	поршень :: положение	перемещение поршня Y (выход)

Матрица A приведена в таблице 6. Её элементы можно связать со схемой. Число -5 в первой строке (столбец x_8) — главная обратная связь: $k_{yc}k_{oc}=50\cdot 0,1$. Число -10 во второй строке (столбец x_5) — механическая обратная связь $k_{мос}$. Единицы в строках x_3, x_5 и x_8 означают, что угол якоря, положение золотника и положение поршня получаются интегрированием соответствующих скоростей. Остальные матрицы: $B=[500\ldots0]^T$, $C=[0\ldots01]$, $D=0$.

Таблица 6 – Матрица A модели ЭГСП

	x_1	**x_2**	**x_3**	**x_4**	**x_5**	**x_6**	**x_7**	**x_8**
x_1	−200	0	0	0	0	0	0	−5
x_2	2,5	−233,3	$-3{,}333\cdot10^5$	0	−10	0	0	0
x_3	0	1	0	0	0	0	0	0
x_4	0	0	$5{,}233\cdot10^7$	−51,23	−400	0	0	0
x_5	0	0	0	1	0	0	0	0
x_6	0	0	0	0	4,3	−3,091	−0,00199	0
x_7	0	0	0	0	0	$6{,}151\cdot10^6$	−1	0
x_8	0	0	0	0	0	0	1	0

Характеристический многочлен матрицы, нормированный к $a_8=1$, и полюса системы:

$$s^8+488{,}66\,s^7+4{,}168\cdot10^5 s^6+9{,}389\cdot10^7 s^5+9{,}374\cdot10^9 s^4+\ldots \quad (21)$$

$$\ldots+1{,}205\cdot10^{12} s^3+5{,}063\cdot10^{13} s^2+1{,}608\cdot10^{15} s+1{,}730\cdot10^{16}, \quad (22)$$

$$s_{1,2}=-117{,}1\pm564{,}1\,j, s_3=-199{,}8, s_{4,5}=-2{,}88\pm110{,}8\,j, \quad (23)$$

$$s_{6,7}=-16{,}3\pm32{,}4\,j, s_8=-16{,}2. \quad (24)$$

Все полюса лежат левее мнимой оси, значит, привод устойчив. Числитель передаточной функции равен $1{,}730\cdot10^{17}$, поэтому $W(0)=1{,}730\cdot10^{17}/1{,}730\cdot10^{16}=10=1/k_{oc}$, и при $U=0{,}01$ В поршень должен прийти в положение $Y=0{,}1$ м. Этого и следует ожидать: в установившемся режиме сигнал датчика $k_{oc}Y$ сравнивается с U. Получить коэффициенты многочлена вручную трудоёмко, в задании к занятию 02 [8] они выводятся символьно в MATLAB. В главе 5 они сравниваются с независимым расчётом в MATLAB: наибольшее расхождение коэффициентов $2{,}9\cdot10^{-14}$.

4 Программная реализация

4.1 Место подсистемы в системе моделирования

Control Lab состоит из веб-интерфейса (React, TypeScript) и сервера (Python, FastAPI). Пользователь собирает схему в браузере и нажимает «Запустить». Интерфейс отправляет схему на сервер, сервер строит модель, рассчитывает по ней графики и характеристики и возвращает результат. Разработанная подсистема выполняет первый шаг этого расчёта: остальные модули сервера работают уже с матрицами A, B, C, D. Файлы подсистемы перечислены в таблице 7.

Таблица 7 – Файлы подсистемы

Файл	Назначение
backend/app/validation/validator.py	проверка схемы (раздел 2.2)
backend/app/simulation/hierarchy.py	раскрытие подсистем (раздел 2.3)
backend/app/simulation/assembly.py	модели звеньев, таблицы соединений, петли без инерции, итоговые матрицы (разделы 2.4–2.6)
backend/app/simulation/model.py	общий порядок: проверка → раскрытие → построение модели
backend/app/analysis/system.py	полюса и передача модели в ответ сервера
frontend/src/components/simulation/ StateSpacePanel.tsx	вкладка «Матрицы» в интерфейсе
backend/scripts/kp_examples.py	схемы примеров главы 3
backend/verification/simulink/kp_simulink.m	схемы примеров в Simulink (глава 5)
backend/verification/*.m	расчёт передаточных функций примеров в MATLAB (глава 5)
backend/scripts/kp_verification.py	сравнение программы с Simulink и MATLAB (глава 5)

4.2 Входные данные

Схема передаётся на сервер в формате JSON: список блоков с типом и параметрами и список соединений. Каждое соединение указывает блок и порт, откуда идёт сигнал (from), и блок и порт, куда он приходит (to). Ниже приведена схема примера 3.1 (координаты блоков на холсте опущены):

```
{
  "blocks": [
    {"id": "r", "type": "StepInput",
     "parameters": {"amplitude": 1.0, "t0": 0.0}},
    {"id": "e", "type": "Sum", "parameters": {"signs": ["+", "-"]}},
    {"id": "pi", "type": "PIDController",
     "parameters": {"kp": 2.0, "ki": 1.0, "kd": 0.0, "filter_n":
100.0}},
```

```
    {"id": "lag1", "type": "FirstOrderLag",
     "parameters": {"k": 1.0, "T": 1.0, "y0": 0.0}},
    {"id": "lag2", "type": "FirstOrderLag",
     "parameters": {"k": 1.0, "T": 0.5, "y0": 0.0}},
    {"id": "y", "type": "Scope", "parameters": {"label": "y"}}
  ],
  "connections": [
    {"from_block": "r", "from_port": "out",
     "to_block": "e", "to_port": "in1"},
    {"from_block": "e", "from_port": "out",
     "to_block": "pi", "to_port": "in"},
    {"from_block": "pi", "from_port": "out",
     "to_block": "lag1", "to_port": "in"},
    {"from_block": "lag1", "from_port": "out",
     "to_block": "lag2", "to_port": "in"},
    {"from_block": "lag2", "from_port": "out",
     "to_block": "e", "to_port": "in2"},
    {"from_block": "lag2", "from_port": "out",
     "to_block": "y", "to_port": "in"}
  ]
}
```

4.3 Основные структуры данных

Основные структуры данных подсистемы приведены в таблице 8.

Таблица 8 – Структуры данных

Структура	Что хранит
BlockRealization	матрицы одного звена A_i, B_i, C_i, D_i и начальное состояние
StateEntry	какому блоку принадлежит переменная состояния и её номер внутри блока
AlgebraicLoop	блоки петли без инерции и её число обусловленности κ
LinearModel	итоговые матрицы A, B, C, D, начальное состояние, список воздействий и осциллографов, найденные петли
ModelAssemblyError	список сообщений, если модель построить нельзя

4.4 Функции построения модели

Главная функция assemble_linear_model (листинг 1 приложения А) проходит по блокам раскрытой схемы, объединяет их матрицы, заполняет таблицы соединений по списку связей и вычисляет итоговые матрицы по формулам раздела 2.6. Модель отдельного звена по его типу и параметрам строит функция block_realization. Уравнения выходов по группам блоков решает и петли проверяет функция _solve_algebraic_equations. Подсистемы раскрывает функция _expand_level.

Вычисления выполняются библиотеками NumPy и SciPy [6, 7]. Из SciPy взяты поиск сильно связных компонент графа, LU-разложение для

решения уравнений петель и перевод передаточной функции в каноническую форму.

4.5 Что получает пользователь

Построенная модель возвращается в ответе сервера вместе с результатами моделирования. В интерфейсе её показывает вкладка «Матрицы» (рисунок 6). В ней выводятся:

– матрицы A, B, C, D, у которых строки и столбцы подписаны именами блоков;

– таблица «Как собрана модель»: какому блоку и какой подсистеме принадлежит каждая переменная состояния;

– найденные петли без инерционных звеньев и их число обусловленности κ;

– размеры модели в строке состояния: n — число состояний, m — входов, p — выходов.

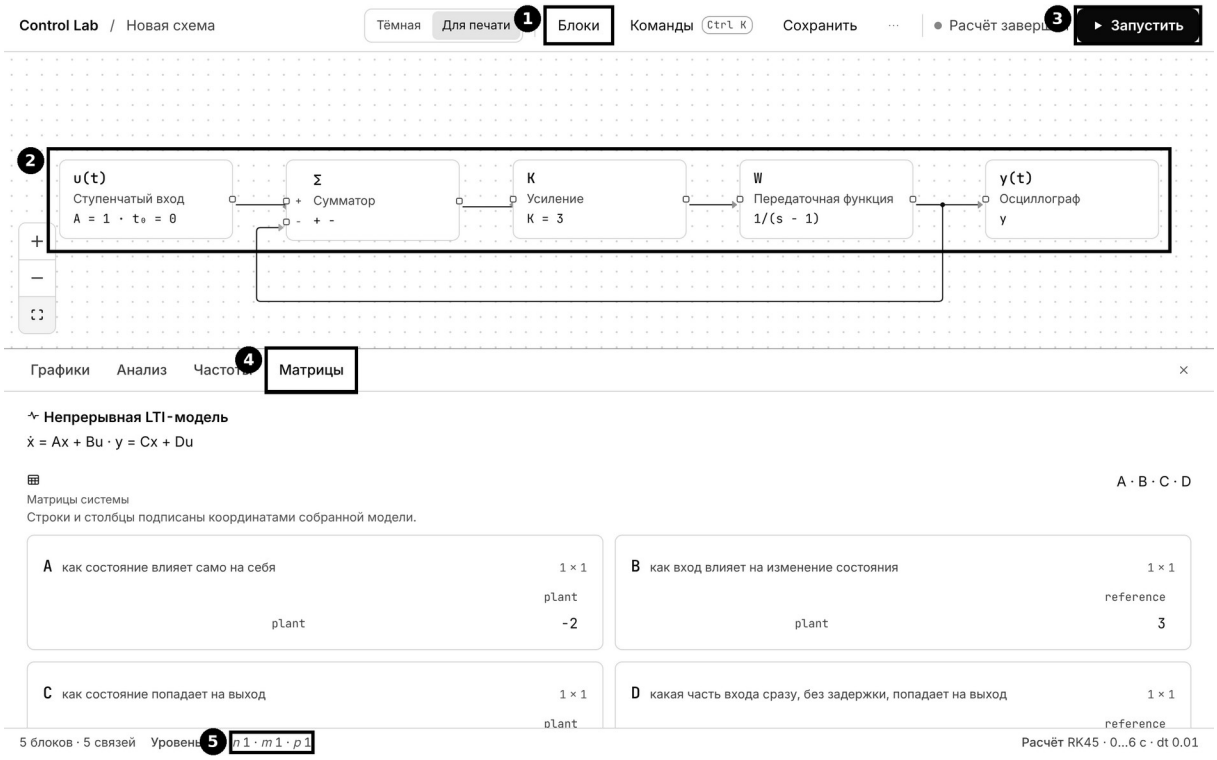


Рисунок 6 – Окно Control Lab: (1) библиотека блоков, (2) схема, (3) запуск расчёта, (4) вкладка «Матрицы», (5) размеры модели

5 Верификация в Simulink и MATLAB

5.1 Как проверяется модель

Каждая схема главы 3 рассчитана ещё двумя способами, не связанными с кодом Control Lab.

Во-первых, та же структурная схема собрана из стандартных блоков Simulink [10]: Step, Sum, PID Controller, Transfer Fcn, Gain, Integrator и осциллограф Scope. Схемы строит скрипт backend/verification/simulink/kp_simulink.m. Он же запускает моделирование методом ode45 с относительной точностью 10^{-10} и сохраняет модели .slx и сигнал осциллографа. Затем этот сигнал накладывается на сигнал осциллографа Control Lab.

Во-вторых, передаточная функция каждой схемы получена в MATLAB сворачиванием структурной схемы функциями tf и feedback [10]. Для ЭГСП порядок сворачивания тот же, что в задании к практическому занятию 02 [8], для перекрёстных связей — с переносом точки съёма сигнала [1]. С этой передаточной функцией сравниваются коэффициенты и полюса, полученные из матриц A, B, C, D программы. Скрипты backend/verification/*.m выполняются и в MATLAB, и в бесплатном GNU Octave [9].

В Control Lab переходный процесс считается так же, как по кнопке «Запустить»: методом RK45 с относительной точностью 10^{-8}, на той же сетке времени, что и в Simulink. Схемы в Simulink показаны на рисунках 7–9, их можно сравнить со схемами главы 3.

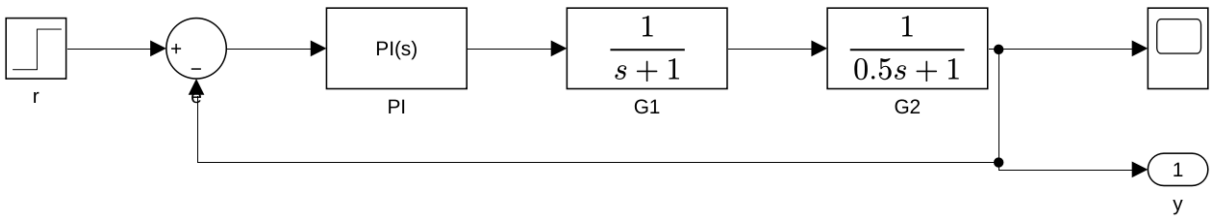


Рисунок 7 – Контур с ПИ-регулятором в Simulink

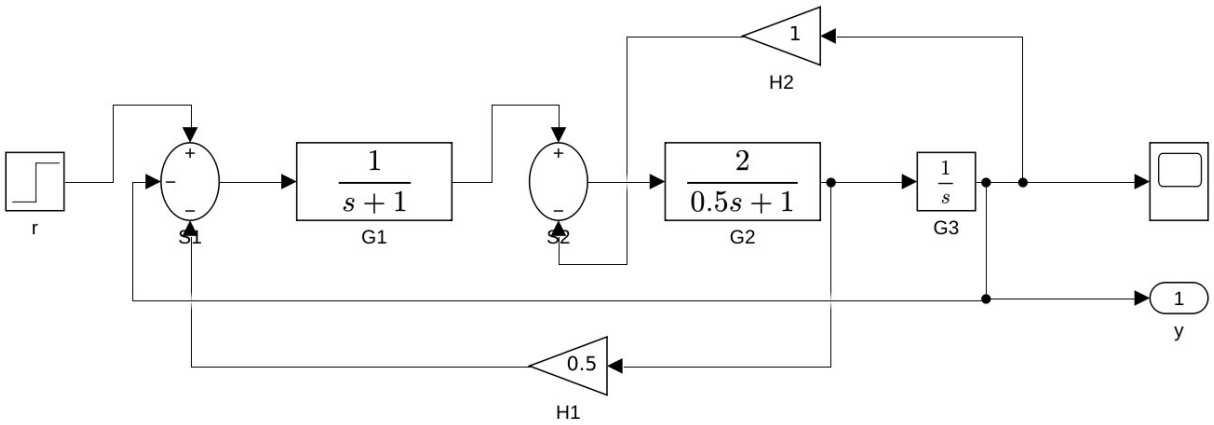


Рисунок 8 – Перекрёстные обратные связи в Simulink

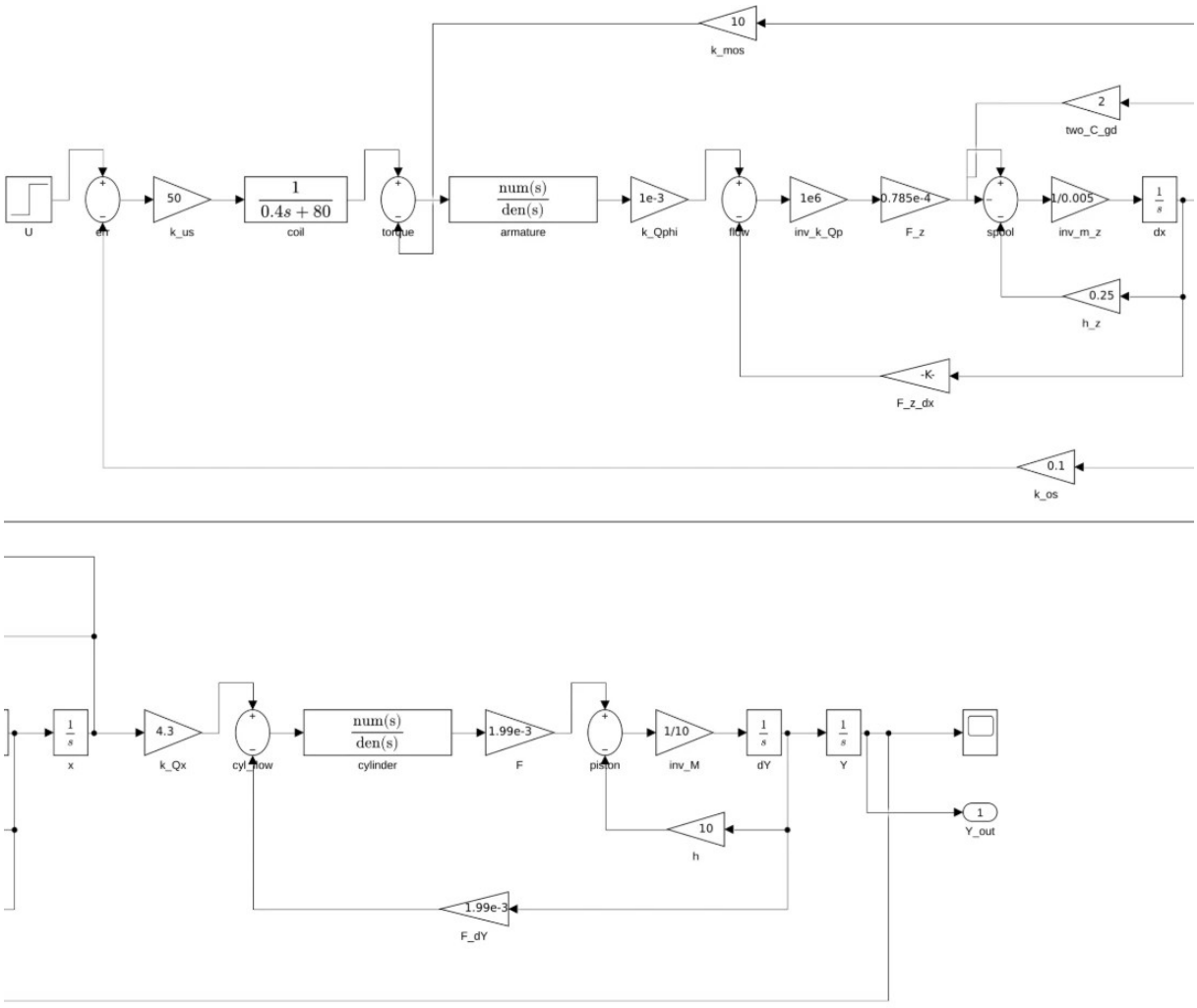


Рисунок 9 – ЭГСП 8-го порядка в Simulink (схема разрезана на две части: вверху — от входа U до золотника, внизу — от золотника до выхода Y)

Сравнение с сохранёнными результатами Simulink запускается одной командой:

```
cd backend
python scripts/kp_verification.py
```

Скрипт расчёта ЭГСП в MATLAB приведён в приложении Б.

5.2 Результаты

Расхождение осциллограмм считается как наибольшая по времени разность сигналов, делённая на наибольшее значение сигнала. Расхождение коэффициентов и полюсов — как наибольшая относительная разность. Результаты приведены в таблице 9.

Таблица 9 – Наибольшие относительные расхождения программы с Simulink и MATLAB

Схема	n	y(t), Simulink	Знаменатель ПФ	Числитель ПФ	Полюса
Контур с ПИ-регулятором	3	$5{,}8\cdot10^{-9}$	$7{,}8\cdot10^{-16}$	$7{,}8\cdot10^{-16}$	$2{,}2\cdot10^{-16}$
Перекрёстные обратные связи	3	$2{,}3\cdot10^{-9}$	$8{,}9\cdot10^{-16}$	$2{,}2\cdot10^{-15}$	$1{,}3\cdot10^{-15}$
ЭГСП 8-го порядка	8	$7{,}0\cdot10^{-9}$	$2{,}9\cdot10^{-14}$	$1{,}0\cdot10^{-14}$	$4{,}7\cdot10^{-14}$

Осциллограммы Control Lab и Simulink расходятся меньше чем на 10^{-8} от размаха сигнала. Такое расхождение ожидаемо: Control Lab считает переходный процесс с относительной точностью 10^{-8}, Simulink — с точностью 10^{-10}, а разница двух численных решений не больше суммы их погрешностей, то есть порядка 10^{-8}. Коэффициенты передаточных функций и полюса совпадают до 13–15 значащих цифр, а это предел точности чисел, с которыми работает компьютер. Значит, модель программы и модель, полученная в MATLAB другим путём, описывают одну и ту же систему. Коэффициенты ЭГСП приведены в таблице 10.

Таблица 10 – ЭГСП: коэффициенты знаменателя передаточной функции

Коэффициент	Программа	MATLAB
a_8	1	1
a_7	488,6569734	488,6569734
a_6	$4{,}168276922\cdot10^{5}$	$4{,}168276922\cdot10^{5}$
a_5	$9{,}388871922\cdot10^{7}$	$9{,}388871922\cdot10^{7}$
a_4	$9{,}373583697\cdot10^{9}$	$9{,}373583697\cdot10^{9}$
a_3	$1{,}204873189\cdot10^{12}$	$1{,}204873189\cdot10^{12}$
a_2	$5{,}062753131\cdot10^{13}$	$5{,}062753131\cdot10^{13}$
a_1	$1{,}608112725\cdot10^{15}$	$1{,}608112725\cdot10^{15}$
a_0	$1{,}730356775\cdot10^{16}$	$1{,}730356775\cdot10^{16}$

5.3 Осциллограммы

На рисунках 10–12 сигнал осциллографа Control Lab (сплошная линия, так его показывает вкладка «Графики») наложен на сигнал осциллографа Scope в Simulink (кружки).

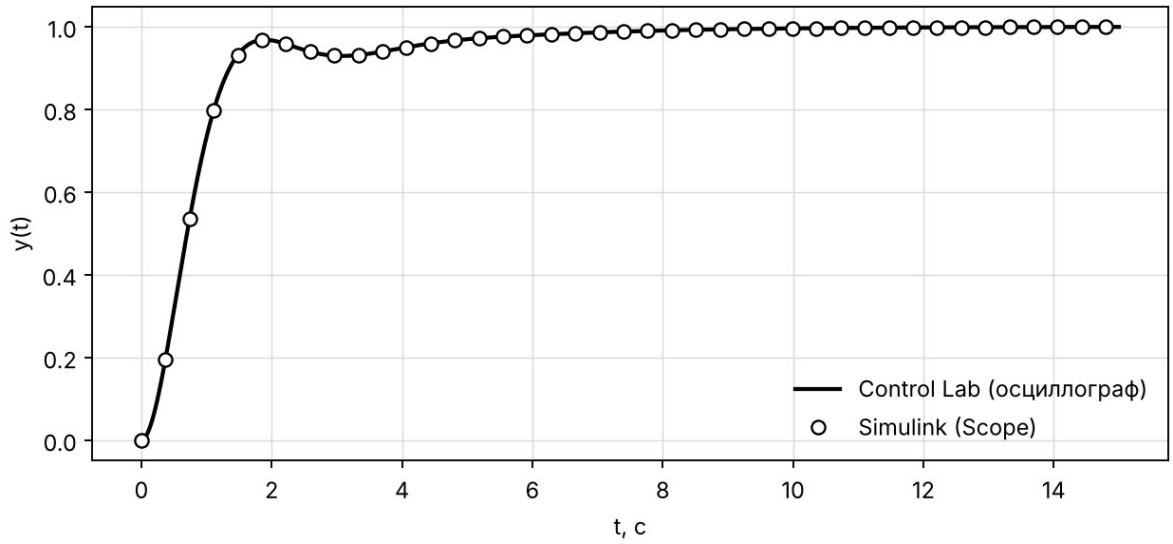


Рисунок 10 – Контур с ПИ-регулятором: Control Lab и Simulink

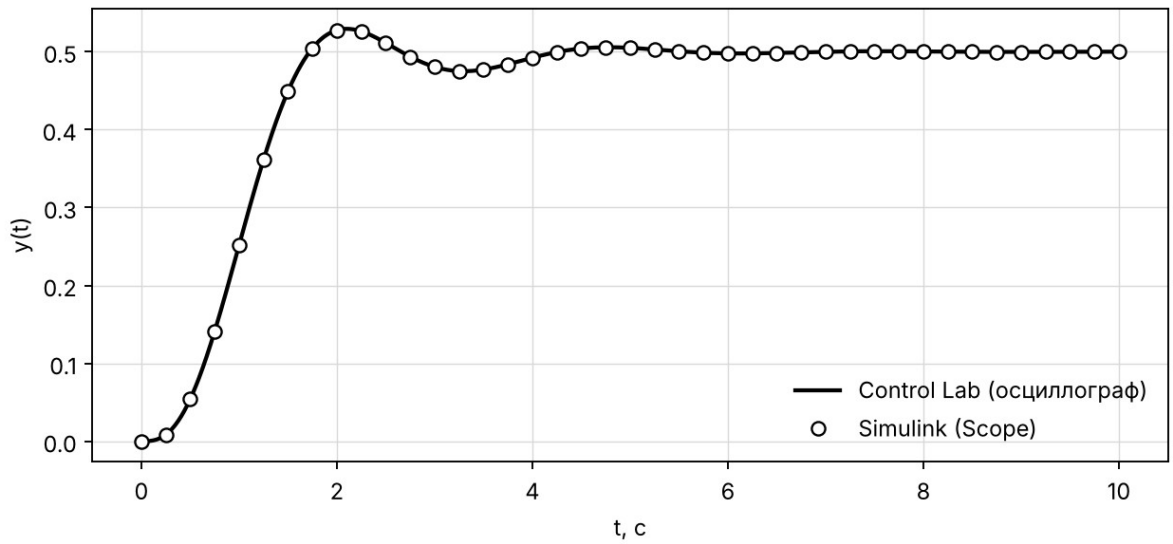


Рисунок 11 – Перекрёстные обратные связи: Control Lab и Simulink

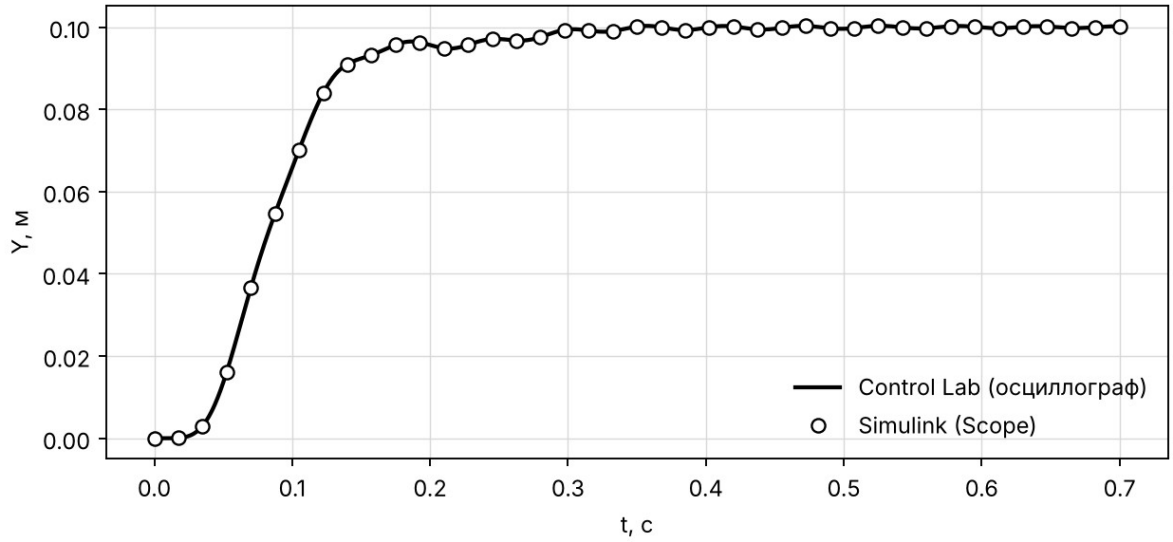


Рисунок 12 – ЭГСП, U = 0,01 В, интервал 0…0,7 с: Control Lab и Simulink

Кривые совпадают на всём интервале. Поршень ЭГСП примерно за 0,3 с приходит в положение 0,1 м, как и следует из $W(0)=10$ (раздел 3.3). Небольшие затухающие колебания после этого дают слабо задемпфированные полюса $-2,88\pm110,8j$: их вещественная часть мала по сравнению с мнимой, поэтому колебания затухают медленно.

6 Руководство пользователя

6.1 Построение модели схемы

Основные элементы окна отмечены цифрами на рисунке 6 (раздел 4.5).

1. Откройте Control Lab в браузере. Выберите готовый пример на пустом холсте или соберите схему из блоков: кнопка «Блоки» (1) открывает библиотеку, блоки перетаскиваются на холст (2), выход блока соединяется со входом другого блока мышью.

2. Нажмите «Запустить» (3) или Ctrl+Enter.

3. Откройте вкладку «Матрицы» (4). В ней показаны матрицы A, B, C, D и таблица «Как собрана модель». Размеры модели выводятся в строке состояния (5).

4. Чтобы изменить параметр, щёлкните блок и введите новое значение в поле (1) на рисунке 13. Модель будет перестроена при следующем запуске.

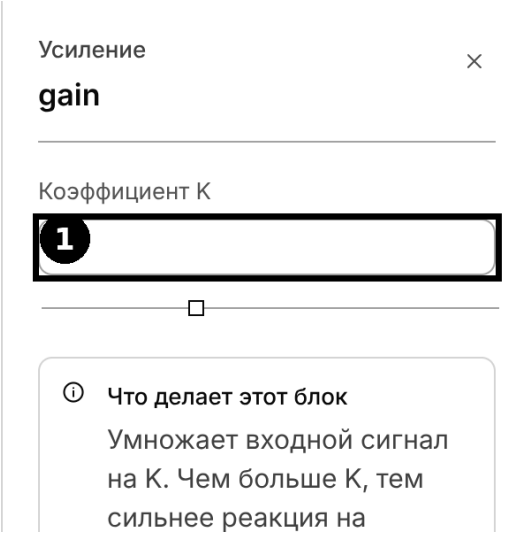


Рисунок 13 – Панель параметров блока: (1) — поле коэффициента

6.2 Работа с подсистемами

Подсистема на схеме отмечена надписью SUB (рисунок 14, отметка 1). Двойной щелчок открывает её содержимое (рисунок 15). После запуска таблица «Как собрана модель» показывает путь подсистемы для каждой переменной (рисунок 16).

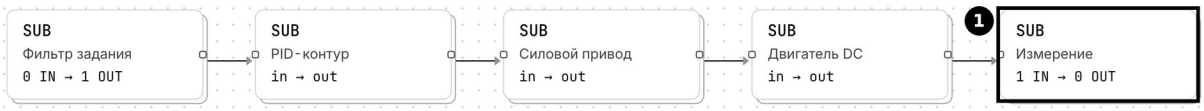


Рисунок 14 – Схема из подсистем: (1) — подсистема «Измерение»

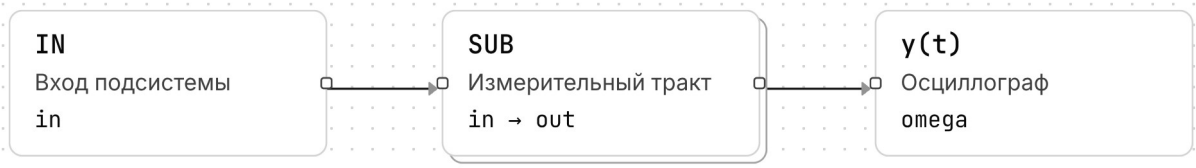


Рисунок 15 – Содержимое подсистемы «Измерение»

Как собрана модель

Координаты x связаны с динамическими блоками исходной схемы.

Координата	Состояние	Блок	Подсистема
x_1	**controller::pid.x1**	controller::pidPIDController	controller
x_2	**controller::pid.x2**	controller::pidPIDController	controller
x_3	**drive::drive_lag**	drive::drive_lagFirstOrderLag	drive
x_4	**motor::motor_model.x1**	motor::motor_modelSecondOrderOscillator	motor
x_5	**motor::motor_model.x2**	motor::motor_modelSecondOrderOscillator	motor
x_6	**measurement::sensor_path::adc_stage::signal_filter::filter**	measurement::sensor_path::adc_stage::signal_filter::filterFirstOrderLag	measurement / s

Рисунок 16 – Таблица «Как собрана модель» для схемы с подсистемами

6.3 Если модель не построена

Если модель построить нельзя, вместо результатов открывается панель «Проблемы» (рисунок 17). Каждое сообщение называет причину и блоки, которые нужно исправить. Чаще всего это неподключённый вход, недопустимый параметр или петля без инерционных звеньев. Петлю можно исправить, изменив её коэффициент или добавив в неё инерционное звено (раздел 2.6).

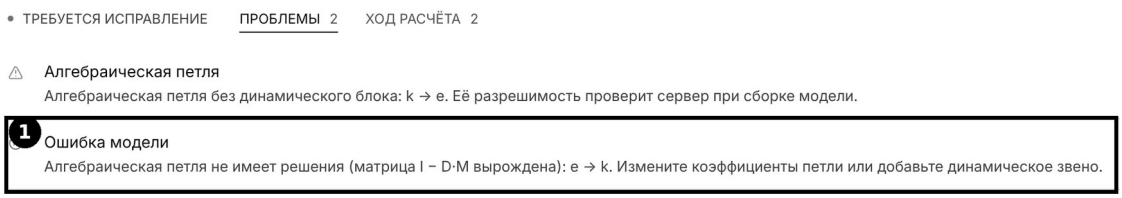


Рисунок 17 – Панель «Проблемы»: петля без инерционных звеньев, которую нельзя решить

Заключение

В курсовом проекте разработана подсистема построения математической модели САУ в пространстве состояний по структурной схеме для веб-системы моделирования Control Lab.

Алгоритм проверяет схему, раскрывает подсистемы, объединяет уравнения звеньев, записывает соединения таблицами M и N и решает уравнения выходов по ходу сигнала. Петли без инерционных звеньев решаются как система линейных уравнений. Если решение петли теряет больше половины верных знаков, модель не строится, и пользователь получает сообщение с указанием блоков петли.

Подсистема работает в серверной части Control Lab. Полученная модель выводится во вкладке «Матрицы» вместе с таблицей, показывающей, какому блоку и какой подсистеме принадлежит каждая переменная состояния.

Работа алгоритма разобрана на трёх схемах: контуре с ПИ-регулятором, схеме с перекрёстными обратными связями и электрогидравлическом следящем приводе 8-го порядка (31 блок после раскрытия подсистем). Для первых двух схем матрицы программы совпали с ручным расчётом. Все три схемы собраны также в Simulink: осциллограммы Control Lab и Simulink совпали с точностью не хуже $7{,}0 \cdot 10^{-9}$ от размаха сигнала. Коэффициенты передаточных функций и полюса совпали с расчётом в MATLAB с относительной погрешностью не более $4{,}7 \cdot 10^{-14}$. Оба расхождения находятся в пределах точности вычислений.

Построенная модель используется в выпускной квалификационной работе: по ней Control Lab выполняет моделирование, анализ устойчивости и частотный анализ.

Список литературы

1. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.

2. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.

3. Ogata K. Modern Control Engineering. 5th ed. – Upper Saddle River: Prentice Hall, 2010. – 894 p.

4. Голуб Дж., Ван Лоун Ч. Матричные вычисления. – М.: Мир, 1999. – 548 с.

5. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. – М.: Вильямс, 2013. – 1328 с.

6. NumPy Documentation [Электронный ресурс]. – URL: https://numpy.org/doc/ (дата обращения: 01.10.2026).

7. SciPy API Reference [Электронный ресурс]. – URL: https://docs.scipy.org/doc/scipy/reference/ (дата обращения: 01.10.2026).

8. Макаренков А.М. Исполнительные устройства систем управления: задания к практическим занятиям 01–03 (моделирование и анализ электрогидравлического следящего привода) [Электронный ресурс]. – Калуга: КФ МГТУ им. Н.Э. Баумана, кафедра ИУК3, 2026.

9. GNU Octave Documentation; Octave control package [Электронный ресурс]. – URL: https://docs.octave.org/latest/, https://gnu-octave.github.io/packages/control/ (дата обращения: 20.10.2026).

10. MATLAB and Simulink Documentation: Control System Toolbox (tf, feedback), Simulink [Электронный ресурс]. – URL: https://www.mathworks.com/help/ (дата обращения: 26.10.2026).

11. Tarjan R. Depth-first search and linear graph algorithms // SIAM Journal on Computing. – 1972. – Vol. 1, No. 2. – P. 146–160.

12. Дорф Р., Бишоп Р. Современные системы управления. – М.: Лаборатория базовых знаний, 2002. – 832 с.

Приложение А. Главный алгоритм

Схема алгоритма приведена на рисунке 18, его реализация — в листинге 1. Функция получает схему, в которой подсистемы уже раскрыты, и возвращает матрицы A, B, C, D.

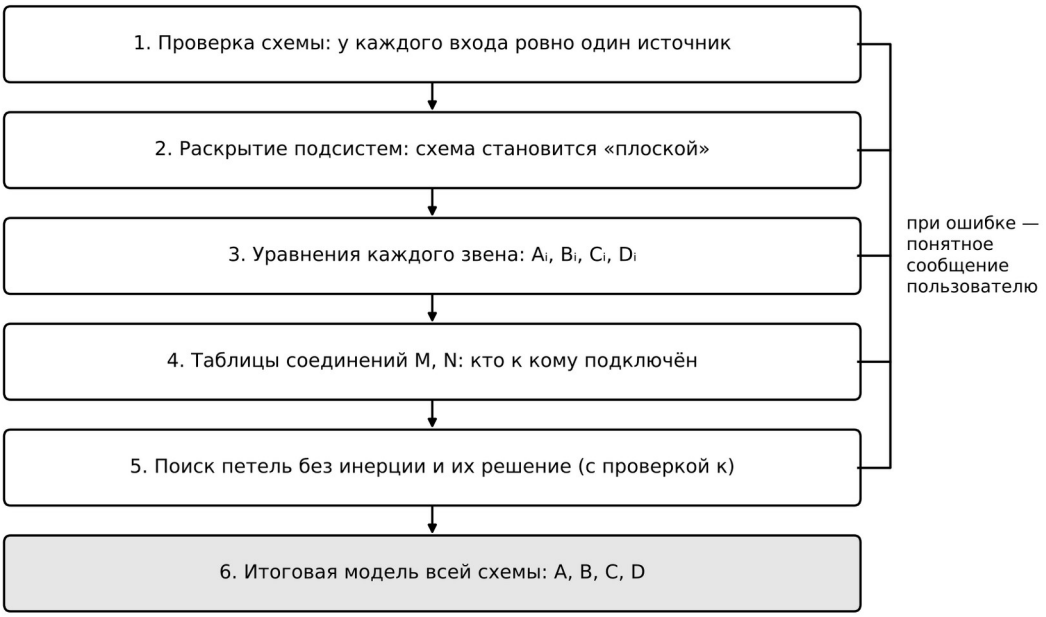


Рисунок 18 – Схема алгоритма построения модели

Листинг 1 – Построение модели схемы (backend/app/simulation/assembly.py)

```
def assemble_linear_model(flat: Diagram) -> LinearModel:
    """Assemble a validated, flattened diagram into (A, B, C, D)."""

    sources: list[StepSource] = []
    scopes: list[ScopeOutput] = []
    source_index: dict[str, int] = {}
    scope_index: dict[str, int] = {}

    dynamic: list[tuple[Block, BlockRealization]] = []
    output_index: dict[str, int] = {}      # block id -> row of y_b
    input_offset: dict[str, int] = {}      # block id -> first row of u_b
    states: list[StateEntry] = []
    total_inputs = 0

    for block in flat.blocks:
        if block.type in SOURCE_TYPES:
            source_index[block.id] = len(sources)
            sources.append(
                StepSource(
                    block.id,
                    get_numeric_parameter(block.parameters, "amplitude", 1.0),
                    get_numeric_parameter(block.parameters, "t0", 0.0),
                )
            )
            continue
        if block.type in SINK_TYPES:
            label = str(block.parameters.get("label") or "").strip() or block.id
            reference = (
                get_numeric_parameter(block.parameters, "reference", 0.0)
```

```
                if "reference" in block.parameters
                else None
            )
            scope_index[block.id] = len(scopes)
            scopes.append(ScopeOutput(block.id, label, reference))
            continue

        realization = block_realization(block)
        if realization.b.shape[1] != len(block.input_ports):
            raise ModelAssemblyError(
                [f"Блок '{block.id}': число входов не совпадает с моделью звена."]
            )
        output_index[block.id] = len(dynamic)
        input_offset[block.id] = total_inputs
        total_inputs += len(block.input_ports)
        for local in range(realization.order):
            label = block.id if realization.order == 1 else f"{block.id}.x{local + 1}"
            states.append(StateEntry(block.id, block.type, local, label))
        dynamic.append((block, realization))

    realizations = [realization for _, realization in dynamic]
    n = sum(r.order for r in realizations)
    n_out = len(dynamic)
    n_src = len(sources)
    p = len(scopes)

    a_d = _block_diagonal([r.a for r in realizations], n, n)
    b_d = _block_diagonal([r.b for r in realizations], n, total_inputs)
    c_d = _block_diagonal([r.c for r in realizations], n_out, n)
    d_d = _block_diagonal([r.d for r in realizations], n_out, total_inputs)
    x0 = np.concatenate([r.x0 for r in realizations]) if realizations else np.zeros(0)

    m = np.zeros((total_inputs, n_out))
    nr = np.zeros((total_inputs, n_src))
    s_y = np.zeros((p, n_out))
    s_r = np.zeros((p, n_src))
    blocks_by_id = {block.id: block for block in flat.blocks}

    for connection in flat.connections:
        target = blocks_by_id[connection.to_block]
        if target.type in SINK_TYPES:
            row_matrix_y, row_matrix_r, row = s_y, s_r, scope_index[target.id]
        else:
            row = input_offset[target.id] +
target.input_ports.index(connection.to_port)
            row_matrix_y, row_matrix_r = m, nr
        if connection.from_block in source_index:
            row_matrix_r[row, source_index[connection.from_block]] = 1.0
        else:
            row_matrix_y[row, output_index[connection.from_block]] = 1.0

    g = d_d @ m
    owners = [block.id for block, _ in dynamic]
    y_solution, loops = _solve_algebraic_equations(g, np.hstack([c_d, d_d @ nr]),
owners)
    y_x, y_r = y_solution[:, :n], y_solution[:, n:]

    return LinearModel(
        a=a_d + b_d @ m @ y_x,
        b=b_d @ (m @ y_r + nr),
        c=s_y @ y_x,
        d=s_y @ y_r + s_r,
        x0=x0,
        states=states,
        sources=sources,
        scopes=scopes,
        algebraic_loops=loops,
    )
```

Приложение Б. Расчёт ЭГСП в MATLAB

Скрипт сворачивает структурную схему ЭГСП (раздел 3.3) и считает переходный процесс; результаты сравниваются с программой в главе 5.

Листинг 2 – Независимый расчёт ЭГСП (backend/verification/egsp.m)

```
% Независимая проверка ЭГСП 8-го порядка (MATLAB / GNU Octave, пакет control).
% Передаточная функция строится последовательным сворачиванием структурной
% схемы, как в практическом занятии 02 по дисциплине «Исполнительные устройства
% систем управления» (рис. 1, приложение 1), но численно.
% Результат пишется в docs/study/octave_egsp.csv для сравнения с программой.
if exist('OCTAVE_VERSION', 'builtin'), pkg load control; end
s = tf('s');
k1 = 50;  a1 = 0.4;  a0 = 80;  b0 = 1;
c2 = 0.3e-6;  c1 = 7e-5;  c0 = 0.1;  d0 = 1;  k4 = 10;
k2 = 1e-3;  k3 = 1/1e-6;  g1 = 0.785e-4;
e2 = 0.005;  e1 = 0.25;  e0 = 2;  f0 = 0.785e-4;  k5 = 4.3;
h1 = 647e-6/(2*1e7);  h0 = 1e-10;  l0 = 1;  q1 = 1.99e-3;
n2 = 10;  n1 = 10;  n0 = 0;  r0 = 1.99e-3;  k6 = 0.1;

W1 = k1;  W2 = b0/(a1*s + a0);  W3 = d0/(c2*s^2 + c1*s + c0);
W4 = k2;  W5 = k3;  W6 = f0/(e2*s^2 + e1*s + e0);  W7 = k5;
W8 = l0/(h1*s + h0);  W9 = r0/(n2*s^2 + n1*s + n0);
W10 = q1*s;  W11 = g1*s;  W12 = k4;  W13 = k6;

W01 = feedback(W5*W6, W11);              % золотник с гидродинамической обратной
связью
W02 = feedback(W3*W4*W01, W12);          % якорь ЭМП с механической обратной связью
W03 = feedback(W8*W9, W10);              % гидроцилиндр с поршнем
W   = minreal(feedback(W1*W2*W02*W7*W03, W13), 1e-9);   % главный контур

[num, den] = tfdata(W, 'v');
k = find(abs(den) > 0, 1);  num = num(k:end)/den(k);  den = den/den(k);
num = num(find(abs(num) > 0, 1):end);
t = (0:0.0005:0.7)';
y = lsim(W, 0.01*ones(size(t)), t);
write_result('../../docs/study/octave_egsp.csv', num, den, pole(W), t, y);
```